Data Serialization

Module 1: Python Foundations

```
import json
row = {"qty": 12}
text = json.dumps(row)
back = json.loads(text)
print(type(back))
```

By the End of Today, You'll Be Able To…

1

Explain why serialization matters

2

Read and write a real file in Python

3

Choose between JSON, CSV, and TXT

You'll Know These by the End of Today

Save

write data

Load

read data

JSON

structured

CSV

rows + columns

TXT

plain text

Serialize

value to file

Reading Recap: Make It Survive

Today's Reading: Data Serialization

A school's weather station lost every reading when its laptop restarted.
Reopen the reading:
Make It Survive: Data Serialization

The Scenario: A Rooftop Weather Station

A Python dict holds each hour's rain, like 1.2 mm at 7:00.

One night the laptop restarts. By morning, every number is gone.

Saved to `rain.json`, it survives. But key `7` loads back as `"7"`.

READING RECAP

Key Terms and Code From the Reading

SERIALIZE: SAVE

`json.dump(rain, f)` writes the dict into the file as JSON text.

DESERIALIZE: LOAD

`json.load(f)` builds a brand-new dict from that text.

FILE MODES

`"w"` empties the file first. `"a"` adds to the end.

JSON, CSV, TXT

JSON for nested data, CSV for a table, TXT for a simple log.

READING CHECK

You save `{6: 0.4, 7: 1.2}` with `json.dump`, then load it back. What does `restored[7]` do?

A Prints `1.2`

B Raises a `KeyError`

C Raises a `TypeError`

ANSWER KEY

Answer Key: Integer Keys Come Back as Text

Correct answer: B. `KeyError`

JSON keys are always strings, so the key `7` comes back as `"7"`. `restored[7]` finds no such key.

READING CHECK

Each day's record holds the date, the rain total, and a list of alerts. Why not one CSV row per day?

A The alerts list won't fit in one flat cell

B CSV can't store numbers

C No problem: CSV stores lists like JSON

ANSWER KEY

Answer Key: Lists Need JSON, Not CSV

Correct answer: A. The alerts list won't fit

A CSV row is flat, with one value per cell. JSON can keep the list of alerts inside each day's record.

Your Questions From the Reading

Turn and talk, 1 min:
One question each.

We take 2 or 3 questions now.

Bigger ones wait for today's matching slide.

IN THE NEWS

A Spreadsheet Renamed a Gene. 704 Papers Shipped It.



Excel silently turned gene SEPT2 into 2-Sep
1 in 5 gene-list papers had errors
The gene committee renamed it SEPTIN2

A file format decides how to read your data. What happens when it guesses wrong, and nothing tells you?

Source: Ziemann, Eren & El-Osta, Genome Biology 17:177, August 23, 2016

Values in Memory Disappear When the Program Closes

While running

```
name = 'Stapler'
count = 12
```

Values in Memory Disappear When the Program Closes

While running

```
name = 'Stapler'
count = 12
```



After closing

Those variables are gone.

Your Program Forgets Everything When It Closes

Lives in memory

- Variables exist while the program runs
- Close it, and every value disappears

Today's move

Make data survive past the close by saving it to a file.

Saving Your Game, and Loading It Back

PLAYING

Your character's live, in-memory state

Saving Your Game, and Loading It Back

PLAYING

Your character's live, in-memory state

SAVING

Flatten that state into a file you can close

Saving Your Game, and Loading It Back

PLAYING

Your character's live, in-memory state

SAVING

Flatten that state into a file you can close

LOADING

Turn the file back into usable state

Saving Your Game, and Loading It Back

PLAYING

Your character's live, in-memory state

SAVING

Flatten that state into a file you can close

LOADING

Turn the file back into usable state

Serialization is the save. Deserialization is loading the save back.

CHECK FOR UNDERSTANDING

Quick check: In the save-game analogy, which step is like deserializing?

A Playing the game

B Saving the game

C Loading a save file

ANSWER KEY

Save/Load Analogy Recall

Correct answer: Loading a save file

Deserializing turns a saved file back into a real, usable value — exactly what loading a save does for your character.

Three Common Formats Have Different Strengths

JSON

Nested, structured data

CSV

Flat, spreadsheet-style rows

TXT

Whatever structure you define

Three Formats, Three Shapes

JSON

```
{"name":"Stapler",
 "count":12}
```

CSV

```
name,count
Stapler,12
```

TXT

```
Stapler: 12
```

(or anything you define)

JSON Mirrors Python's Own Dicts and Lists

JSON object

looks like a Python dict

```
{ key: value }
```

JSON array

looks like a Python list

```
[ item, item ]
```

Python's `json` module converts between them.

What `import json` Actually Imports

Module

One `.py` file of reusable code. `csv.py` is one.

Package

A folder of modules. `json` is one. Later, `pandas` too.

The dot

`json.dump` means the `dump` function inside `json`.

Every `.py` file you write is a module too.

CODE ALONG

Open the Code-Along

Open `Module1_Lesson8_data-serialization-why-json-csv-and-txt-exist_STUDENT.py`
Run it after every section: `python3` + the file name

Sections, in order:

1. Writing to a File
   Reading about it is one thing. Watching a file actually appear on disk is what makes it real.
2. Reading It Back
3. A CSV File Is Just Plain Text
4. Serializing a Record With `json.dump()`
   JSON isn't just readable by a program: it mirrors Python's own dicts and lists almost exactly.
5. Deserializing It Back With `json.load()`

Writing to a File

```python
with open("notes.txt", "w") as f:
    f.write("Restock staplers\n")
    f.write("Restock printer paper\n")
```

Reading It Back

python

```python
with open("notes.txt") as f:
    print(f.read())
```

A CSV File Is Just Plain Text

python

```
csv_content =
"name,count,department\nStapler,12,Office\nPaper,200,
Office\n"

with open("supply.csv", "w") as f:
    f.write(csv_content)

with open("supply.csv") as f:
    print(f.read())
```

Serializing a Record With `json.dump()`

```python
import json

supply = {"name": "Stapler", "count": 12,
"department": "Office"}

with open("supply.json", "w") as f:
    json.dump(supply, f)
```

Deserializing It Back With `json.load()`

```python
import json

with open("supply.json") as f:
    supply = json.load(f)

print(supply["name"])
print(type(supply))
```

CODE-ALONG DONE

Back to the Slides: Review

You just typed it and ran it. Now check that it stuck, without the code in front of you.

QUICK CHALLENGE

If you open `notes.txt` again in `"w"` mode and write one new line, what happens?

A Old lines are gone — `"w"` replaces the file

B Old lines stay; new line is added

C Python raises an error

ANSWER KEY

Write Mode Overwrites

Correct: `"w"` replaces the whole file

`"w"` mode starts the file empty before writing. Appending requires `"a"` mode instead.

QUICK CHALLENGE

You serialize a Python tuple to JSON, then load it back. What type is it?

A `tuple`, unchanged

B `list`

C `dict`

ANSWER KEY

JSON Has No Tuple Type

Correct answer: `list`

JSON has objects and arrays, but no tuple type. A Python tuple becomes a JSON array and loads back as a Python list.

Watch It Run: A Tuple's JSON Round Trip

Step 1 of 5

row is a tuple: ("Stapler", 12)

```
import json
row = ("Stapler", 12)
text = json.dumps(row)
print(text)
back = json.loads(text)
print(type(back))
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

AI Tools Constantly Read and Write JSON

APIs

send structured requests and responses

Configs

store settings machines can read

AI responses

often return structured JSON

Understanding the format lets you catch mistakes instead of guessing.

TURN AND TALK

Two entries both use the key `"Stapler."` What happens when you load it?

```
{
  "Stapler": 12,
  "Stapler": 7
}
```

Discuss

- Which value survives?
- Why is a duplicate key risky?
- How would you restructure the data?

TIP

“JSON or plain text for a batch file? Why?”

JSON

Structured records; explicit fields

TXT

Simple `ADD` / `REMOVE` / `SEARCH` lines

Answer like an interview, with PREP: point, reason, example, point.

MODEL ANSWER

Model Answer: PREP

POINT

Plain TXT, with one ADD, REMOVE or SEARCH command per line.

REASON

Each line is one simple command, easy to write and loop over.

EXAMPLE

`read_batch_file` splits each line; `export_to_json` saves the results.

POINT, RESTATED

TXT for simple commands; JSON once records have real structure.

Key Terms — Recap

Save

write data to a file

Load

read file data

Serialize

Python value to saved format

Deserialize

saved format to Python value

JSON

nested structured data

CSV / TXT

flat rows / plain text

So Far, We've Covered…

Why it matters

You decide when text becomes a date, not the tool.

Next

Writing your first function from scratch.

Looking Ahead

Next class

Writing a function from scratch.